

Épreuve de Programmation Orientée Objet 2

Licence Informatique — Sujet d'entraînement C

Decorator et Command/MVC (avec annulation)

Juin 2026

Durée : 2 heures. **Documents autorisés :** documentation des classes conteneur (livret distribué) et de l'interface `JAVA Iterator`.

Ce sujet est un sujet d'entraînement généré à partir du contenu du cours de POO2 (Iterator, Singleton/Enum, Strategy, Decorator, Command/MVC, Builder). Il n'a pas valeur de sujet officiel mais reprend le format et l'esprit des épreuves de M. Champesme.

Ce qui est évalué dans cette épreuve

Dans cette épreuve sont évaluées vos capacités à : - utiliser correctement le langage JAVA pour implémenter une classe ; - effectuer des choix corrects pour la visibilité des champs et des méthodes ; - respecter le contrat hérité des super-classes et des interfaces lors de la définition d'une nouvelle classe ; - donner l'implémentation d'une classe en conformité avec son interface et son contrat ; - utiliser correctement l'héritage et la généricité pour la définition de classes ; - respecter la séparation des responsabilités de l'architecture MVC ; - mettre en œuvre les design patterns **Decorator**, **Command** et l'architecture **MVC**.

1 Design pattern *Decorator*

Exercice 1 On souhaite pouvoir suivre (logger) les itérations effectuées sur une collection, sans modifier la classe d'origine, en appliquant le design pattern *Decorator*.

- Donnez le code source complet d'une classe générique `IterateurTrace<E>` qui implémente `Iterator<E>` en décorant un `Iterator<E>` existant fourni au constructeur. Chaque appel à `next()` doit afficher sur la sortie standard un message de la forme `"-> element retourne : <element>"` avant de renvoyer l'élément. Les méthodes `hasNext()` et `remove()` doivent être simplement déléguées, sans aucune trace affichée.
- On veut maintenant pouvoir décorer une `Collection<E>` tout entière, de sorte que sa méthode `iterator()` renvoie automatiquement un `IterateurTrace<E>` enveloppant l'itérateur réel de la collection, alors que toutes les autres méthodes ont exactement le même comportement que sur la collection d'origine. Donnez le code source complet d'une classe générique `CollectionTracee<E>` qui implémente `Collection<E>` selon ce principe. Vous implémenterez `size()`, `isEmpty()`, `add(E)` et `iterator()` en les déléguant à la collection décorée ; pour toute autre méthode de l'interface `Collection<E>`, vous lancerez une `UnsupportedOperationException`.
- Quelle est la différence essentielle, du point de vue de la structure du code, entre ce décorateur et un décorateur qui empêcherait toute modification (comme `UnmodifiableCollection`) ? Répondez en deux ou trois lignes.

IMPORTANT : pour ces classes, les commentaires et assertions ne sont pas demandés. Toute méthode de `Collection<E>` non explicitement listée à la question 2 doit lancer `UnsupportedOperationException`.

2 Design pattern *Command* et architecture *MVC*

Exercice 2 On reprend le contexte d'un jeu d'aventure textuelle. On suppose qu'une classe `Inventaire` existe déjà, possédant des méthodes `ajouter(ObjetZork o)` et `retirer(ObjetZork o)`. On souhaite maintenant que chaque

commande exécutée par le joueur puisse être **annulée** (fonctionnalité « undo »).

1. Donnez le code source complet d'une interface `Commande` possédant **deux** méthodes abstraites : `executer()`, qui exécute l'action représentée, et `annuler()`, qui défait l'action précédemment réalisée par `executer()`.
2. Donnez le code source complet d'une classe `CommandeRamasserObjet` qui implémente `Commande`. Le constructeur reçoit une `Salle`, un `Inventaire`, un `ObjetZork` et une `Vue`. La méthode `executer()` doit retirer l'objet de la salle, l'ajouter à l'inventaire, puis demander à la vue d'afficher "Vous avez ramasse <nom de l'objet>.". La méthode `annuler()` doit faire exactement l'inverse (retirer l'objet de l'inventaire, le remettre dans la salle), puis demander à la vue d'afficher "Vous reposez <nom de l'objet>.".
3. Donnez le code source complet d'une classe `Contrôleur` qui maintient une **pile** des commandes exécutées, avec :
 - une méthode `executer(Commande commande)` qui exécute la commande puis l'empile ;
 - une méthode `annulerDerniere()` qui, s'il existe au moins une commande dans la pile, en dépile la dernière et appelle `annuler()` dessus (ne fait rien si la pile est vide).
4. En deux ou trois lignes, expliquez pourquoi il est essentiel que `annuler()` soit définie au niveau de l'interface `Commande` elle-même, plutôt que d'être ajoutée seulement à certaines classes concrètes qui en auraient besoin, du point de vue de la classe `Contrôleur`.

IMPORTANT : pour ces classes, les commentaires et assertions ne sont pas demandés.

Sujet d'entraînement généré par Claude — Anthropic.